Day 4: Histograms & Thresholding

Today we will cover:

- Image Histograms
- Global Thresholding
- Adaptive Thresholding
- Otsu's Method
- Applications
- Lab Exercise
- Mini-Project

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

# Load grayscale image
img = cv2.imread('sample.jpg', 0)    # replace with your image file

plt.imshow(img, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis('off')
plt.show()
```



Image Histograms

- Show distribution of pixel intensities.
- Useful for analyzing contrast and brightness.

```
plt.hist(img.ravel(), 256, [0,256])
plt.title("Image Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.show()
```

Global Thresholding

- Apply a single threshold value to the entire image.

```
_, thresh_global = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

plt.imshow(thresh_global, cmap='gray')
plt.title("Global Thresholding (127)")
plt.axis('off')
plt.show()
```



Adaptive Thresholding

- Threshold value is calculated for smaller regions.
- Useful for images with varying illumination.

```
thresh_adaptive = cv2.adaptiveThreshold(img, 255,
                                        cv2.ADAPTIVE_THRESH_MEAN_C,
                                        cv2.THRESH_BINARY, 11, 2)

plt.imshow(thresh_adaptive, cmap='gray')
plt.title("Adaptive Thresholding")
plt.axis('off')
plt.show()
```



Otsu’s Method

- Automatically calculates optimal threshold value.
- Minimizes intra-class variance.

```
_, thresh_otsu = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)

plt.imshow(thresh_otsu, cmap='gray')
plt.title("Otsu's Thresholding")
plt.axis('off')
plt.show()
```



Applications

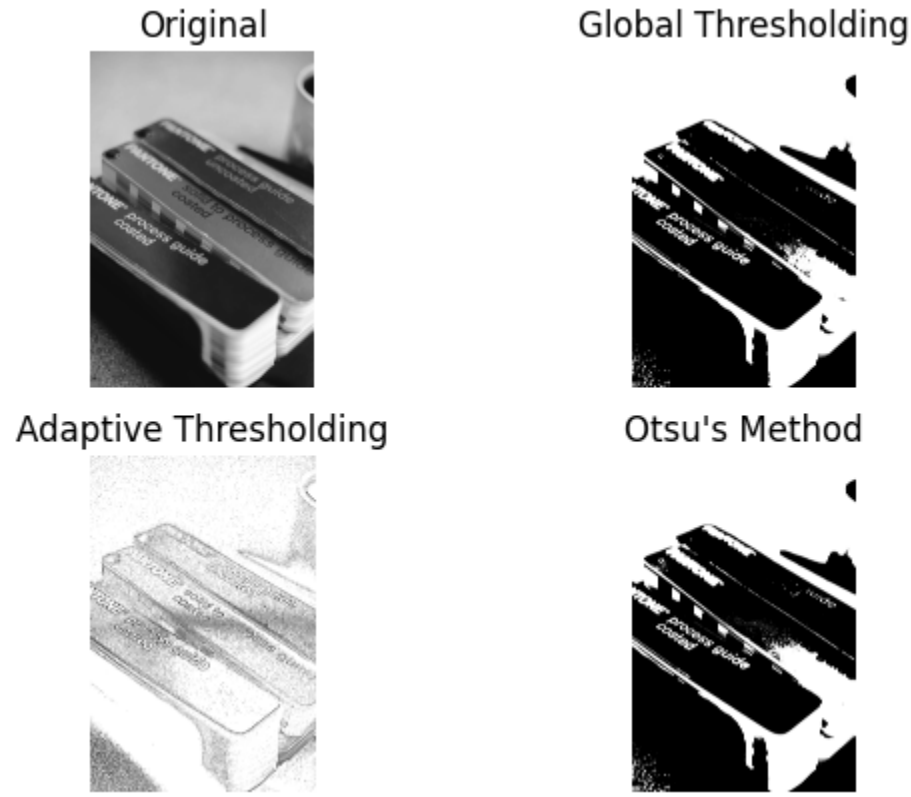
- Document scanning (binarization)
- Medical imaging (tumor segmentation)
- Industrial inspection (defect detection)

Lab Exercise

- Compare global, adaptive, and Otsu thresholding on the same image.
- Observe differences in output.

```
titles = ['Original', 'Global Thresholding', 'Adaptive Thresholding', "Otsu's
Method"]
images = [img, thresh_global, thresh_adaptive, thresh_otsu]

for i in range(4):
    plt.subplot(2,2,i+1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis('off')
plt.show()
```



Mini-Project: Document Binarizer

- Input: scanned document image.
- Output: clean binary version using Otsu's method.

```
doc = cv2.imread('sample.jpg', 0)  # replace with your document image
_, doc_bin = cv2.threshold(doc, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

plt.imshow(doc_bin, cmap='gray')
plt.title("Binarized Document")
plt.axis('off')
plt.show()
```

Binarized Document

